

Data Flow Architecture

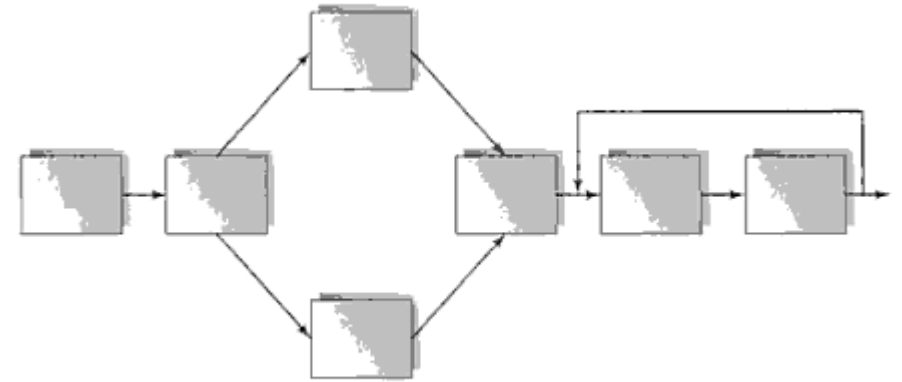
Lecture 5

Data-Flow Architecture - Overview

- Entire software system is viewed as a series of transformation on successive sets of data.
- Data sets and operations are independent of each other.
- Software system is decomposed into data processing elements , where data directs and controls the order of data computation.
- Each component transforms its input data into corresponding output data.

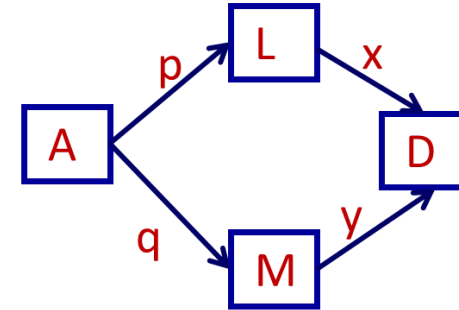
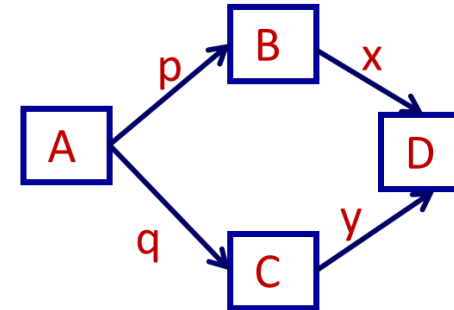
A Sample block diagram for Data-Flow Architecture

- Data move from one sub system to another.
- Data flow can be represented using a graph topology, with cycles, linear structure without cycles, or tree topology.
- Sub systems are independent to each other.
- Relations exist between elements input/output data connections.



A Sample block diagram for Data-Flow Architecture

- Independent sub systems means one sub system can be replaced by another system provided input and output relations are compatible.
- Since sub systems are independent to each other modifiability and reusability are important quality attributes.



Data-Flow Architecture

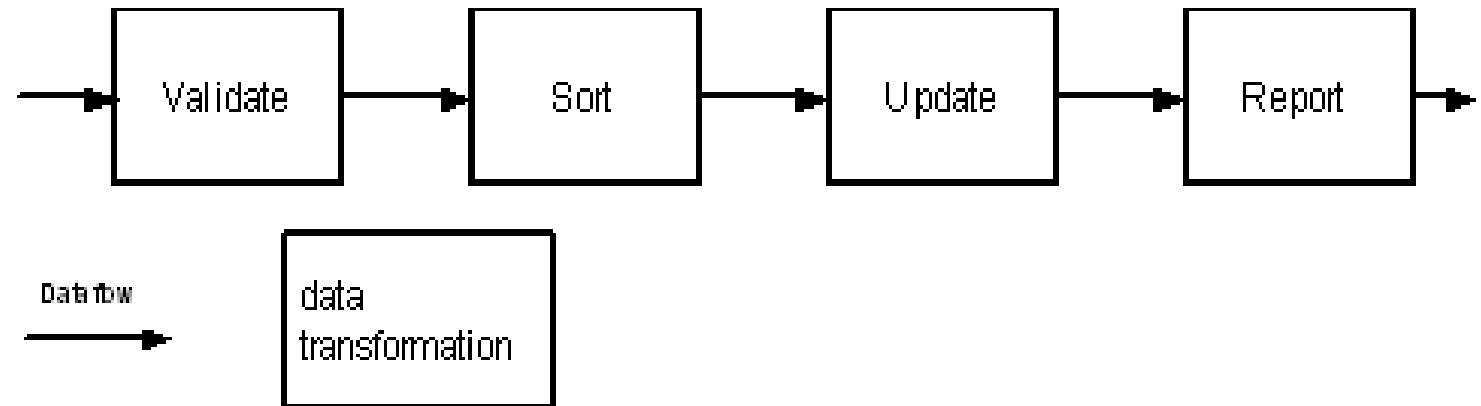
- There are different ways to connect output data of a module to a input of another module .
- Three categories
 1. Batch Sequential
 2. Pipe and Filter (non sequential)
 3. Process Control
- This architecture can be used when there is a well defined series of data transformations (computations) with well defined input, output without user interactions

1. Batch Sequential Systems (1)

- A transformation subsystem or module **cannot** start its process until the previous module completes its computation.
- Data flow carries a batch of data as a whole from a module to next.
- Series of transformation on successive sets of data.
- Data sets and operations are independent of each other.
- Applications:
 - Business data processing in banking
 - Utility billing (telephone, water bills)
 - Compilers
 - ETL (Extract, Transform, Load) tasks in data warehousing
 - In Image Processing for Converting or filtering batches of images
 - Financial Reporting: Monthly payroll generation or end-of-day banking reconciliations

1. Batch Sequential Systems (2)

- Subsystem “Validate” the transaction request (Insert, Delete and update)
- Subsystem “Sort” all the transaction records in ascending order on the primary key
- “Update” module, update the master file
- “Report” module generate a new list



1. Batch Sequential Systems (3)

- In Batch Sequential Systems
 - a number of programs are executed sequentially
 - each program runs to completion
 - data is transmitted between programs via files stored on external storage
 - a program reads data from one or more files,
 - processes it
 - and writes one or more files
 - Connections are established through intermediate files.
 - After exiting, the next program is started, if not the last one.

1. Batch Sequential Systems (4) – Java Implementation

```
def validate(in_file, validated_file, rejected_file):
```

```
    # Placeholder logic for validation
    with open(in_file, 'r') as infile, \
        open(validated_file, 'w') as valid_out, \
        open(rejected_file, 'w') as reject_out:
```

```
    for line in infile:
```

```
        if "VALID" in line: # Sample condition
```

```
            valid_out.write(line)
```

```
        else:
```

```
            reject_out.write(line)
```

```
def sort(input_file, output_file):
```

```
    # Placeholder logic for sorting
    with open(input_file, 'r') as infile:
        lines = infile.readlines()
    lines.sort()
    with open(output_file, 'w') as outfile:
        outfile.writelines(lines)
```

```
def main():
```

```
    validate("TransactionFile.txt", "ValidatedTransactionFile.txt",
            "RejectedTransactionFile.txt")
    sort("ValidatedTransactionFile.txt", "SortedTransactionFile.txt")
    # Additional steps can be added here
```

```
if __name__ == "__main__":
```

```
    main()
```

1. Batch Sequential Systems (5)

- Benefits

- Simple division on subsystems
- Supports easy reuse of components
- Easy to maintain by adding/removing components
- Each subsystem can be stand-alone program
- Low memory (primary memory) requirements

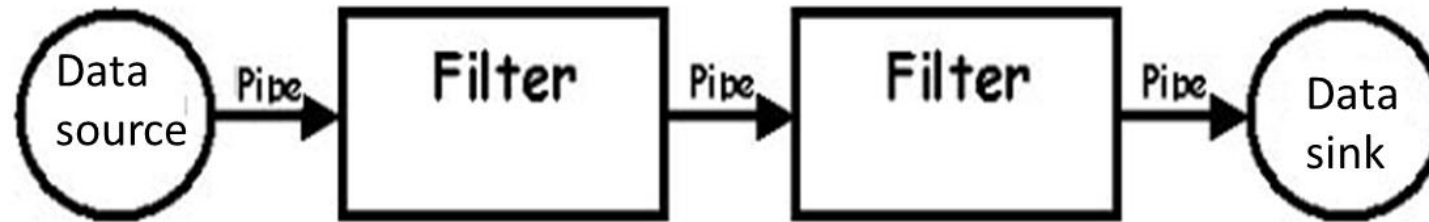
- Limitations

- Implementation requires external control
- No interactive interface (no user involvement)
- Each component has to wait for the output of the previous component to begin (concurrency is not supported, hence low throughput)
- High latency (delay) for I/O

2. Pipe and Filter Architecture (1)

- The system consists of multiple independent components (filters) that process data, connected by pipes.
- Filters can execute independently and do not always need sequential execution.
- A system consists of
 - Data source
 - Filters
 - Pipes
 - Data sinks

2. Pipe and Filter Architecture (2)



- System consists of number of components (filters) that transform data, before passing it on via connectors (pipes) to other components.

2. Pipe and Filter Architecture (3)

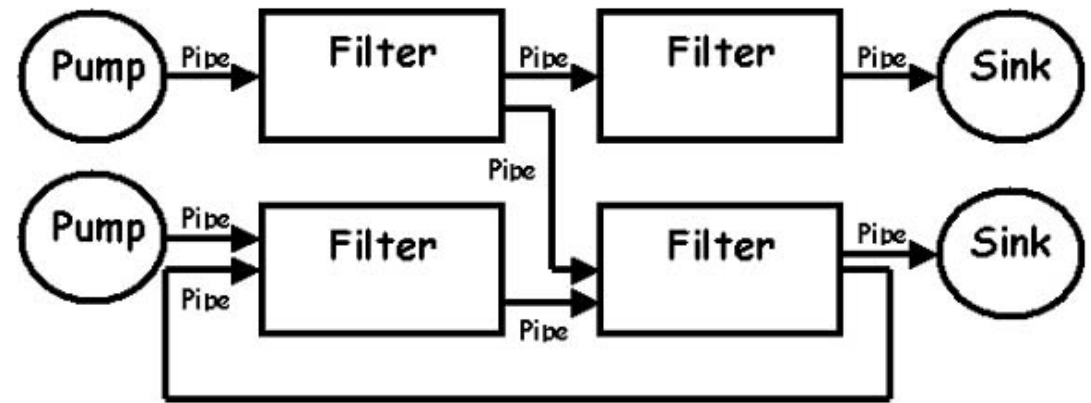
- Data source: Provide the input data to the system
 - Example: File, Database or a keyboard input device
- Filters
 - Processing units that simply processes (transforms or filter) input data
 - A filter can have any number of input pipes and any number of output pipes
- Pipes
 - Connectors that link data sources to the first filter, between filters, and to the data sink
- Data sinks
 - Gather data at the end of a pipeline
 - Example: File, Database, or Computer screen

2. Pipe and Filter Architecture (4)

- Connections between components are data streams
- They can be stream characters, XML record or any other type (12345, AAA BBB, “No 43, Galle Road, Ratmalana”.....)
- Filters read data from its input stream, processes it, and writes it over a pipe for next filter to process
- A filter processes data once received, does not wait until the whole data is received and waits if data are not available at the input (incremental processing)

2. Pipe and Filter Architecture (5) – How it works?

- Filters are independent of each other.
- Pipes are directional stream of data implemented by a data buffer to store all data, until the next filter has time to process it.
- A filter only knows its connected pipes, does not know what are at the other end of the pipe.



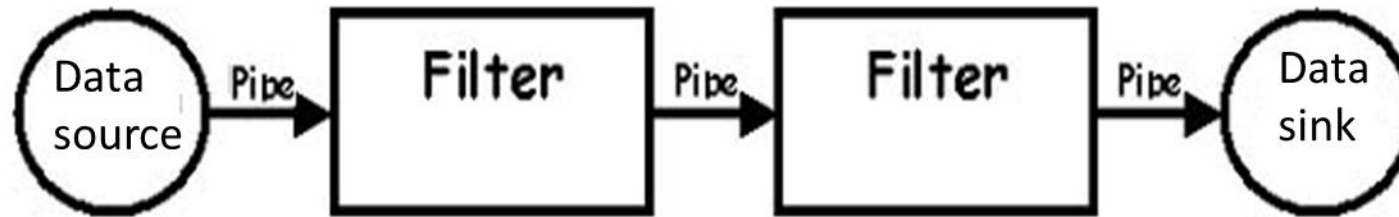
2. Pipe and Filter Architecture (6) – How it works?

- Pure data-driven interaction
- Filters are independent programs that can be combined freely to build a system
- All filters are processes that run (virtually) at the same time (concurrently) on partial data
- Filters can run as different threads in a same program, or processes in same machine or processes in different machines (distributed system)

2. Pipe and Filter Architecture (7) – How it works?

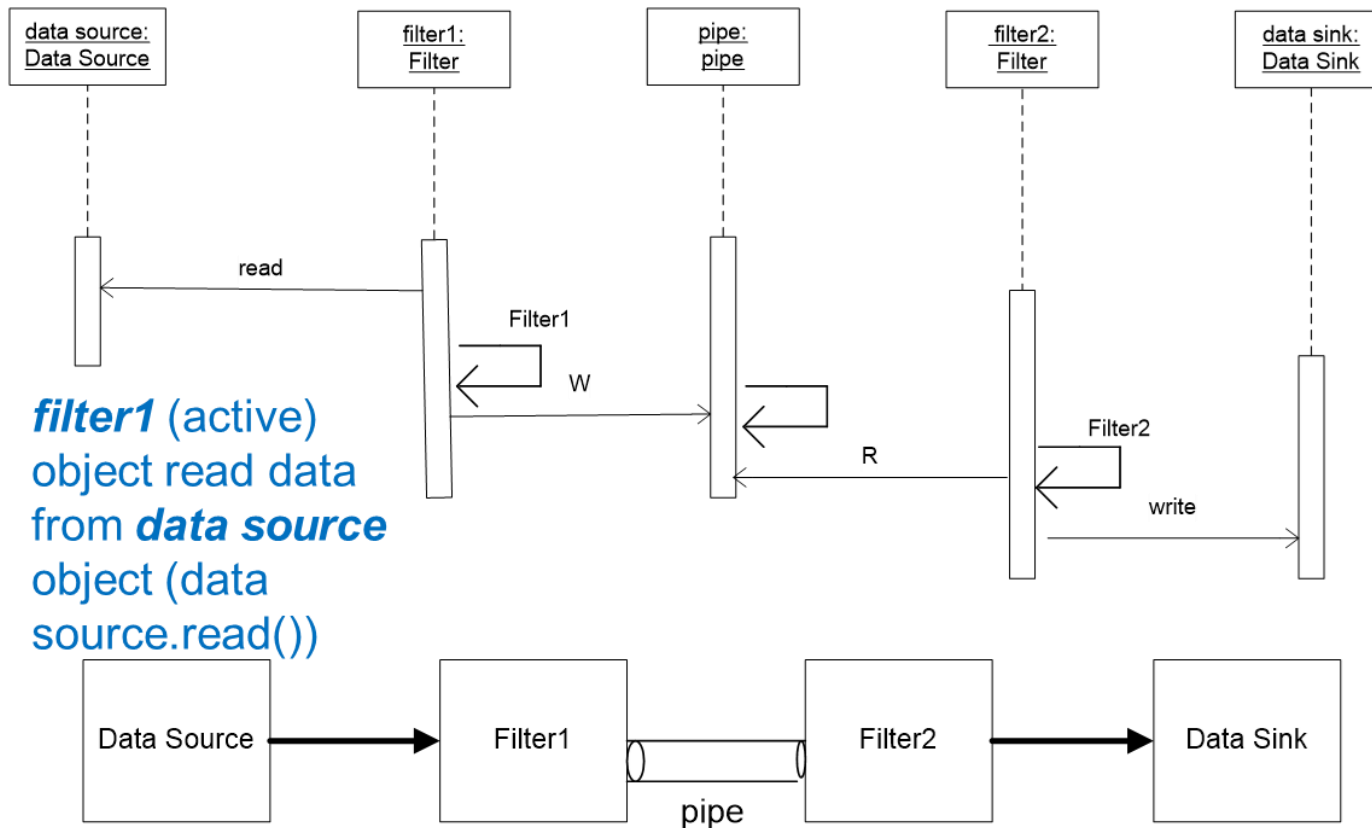
- Active filter
 - Pulls in (read) data from upstream and pushes out (write) process data to downstream. (Filter is active in reading, writing and processing)
- Passive filter
 - It lets connected upstream pipes (active pipe) push (write) data to it and lets downstream pipes to pull (read) data from it.
 - Focus on processing

2. Pipe and Filter Architecture (8) – How to make data flow

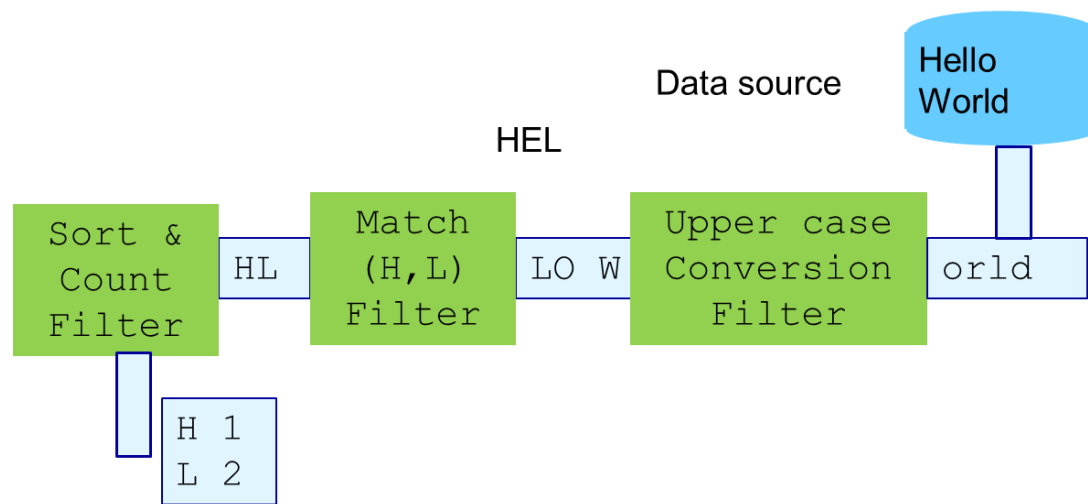


- Push (Write) only: A data source or filter may push data in a downstream
- Pull (Read) only: A data sink or filter may pull data from a upstream
- Push/Pull (Read/Write): A filter may pull data from a upstream or push data in a downstream

2. Pipe and Filter Architecture (10) – UML Sequence Diagram

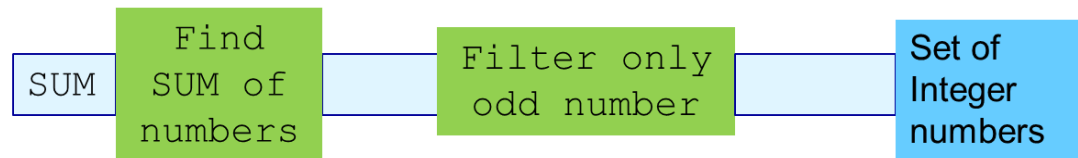


2. Pipe and Filter Architecture (11) – Example



- “Upper case Conversion” filter converts all lower case letters to upper case
- “Match” filter select ‘H’ & ‘L’ letters in the input stream
- “Sort & Count” filter counts number of H’s and L’s

2. Pipe and Filter Architecture (12) – Example



- Input: Data file contain set of integers
- Filter 1: Filter only odd numbers
- Filter 2: Find cumulative summation
- Output: SUM

2. Pipe and Filter Architecture (13) – Java Implementation

```
import threading
import queue
import time

# Filter1: Filters odd numbers and pushes into the pipe
class Filter1(threading.Thread):
    def __init__(self, output_pipe):
        super().__init__()
        self.output_pipe = output_pipe

    def run(self):
        for num in range(1, 11): # Example range 1–10
            if num % 2 == 1:
                self.output_pipe.put(num)
                time.sleep(0.1) # Simulate work
        self.output_pipe.put(None) # Signal end of data
```

```
# Filter2: Reads numbers and computes cumulative sum
class Filter2(threading.Thread):
    def __init__(self, input_pipe):
        super().__init__()
        self.input_pipe = input_pipe

    def run(self):
        cumulative = 0
        while True:
            num = self.input_pipe.get()
            if num is None:
                break
            cumulative += num
            print(f"Received: {num}, Cumulative Sum: {cumulative}")
```

2. Pipe and Filter Architecture (13) – Java Implementation

Pipe (acts like a connection between filters)

pipe = queue.Queue()

Create and start filters

f1 = Filter1(pipe)

f2 = Filter2(pipe)

f1.start()

f2.start()

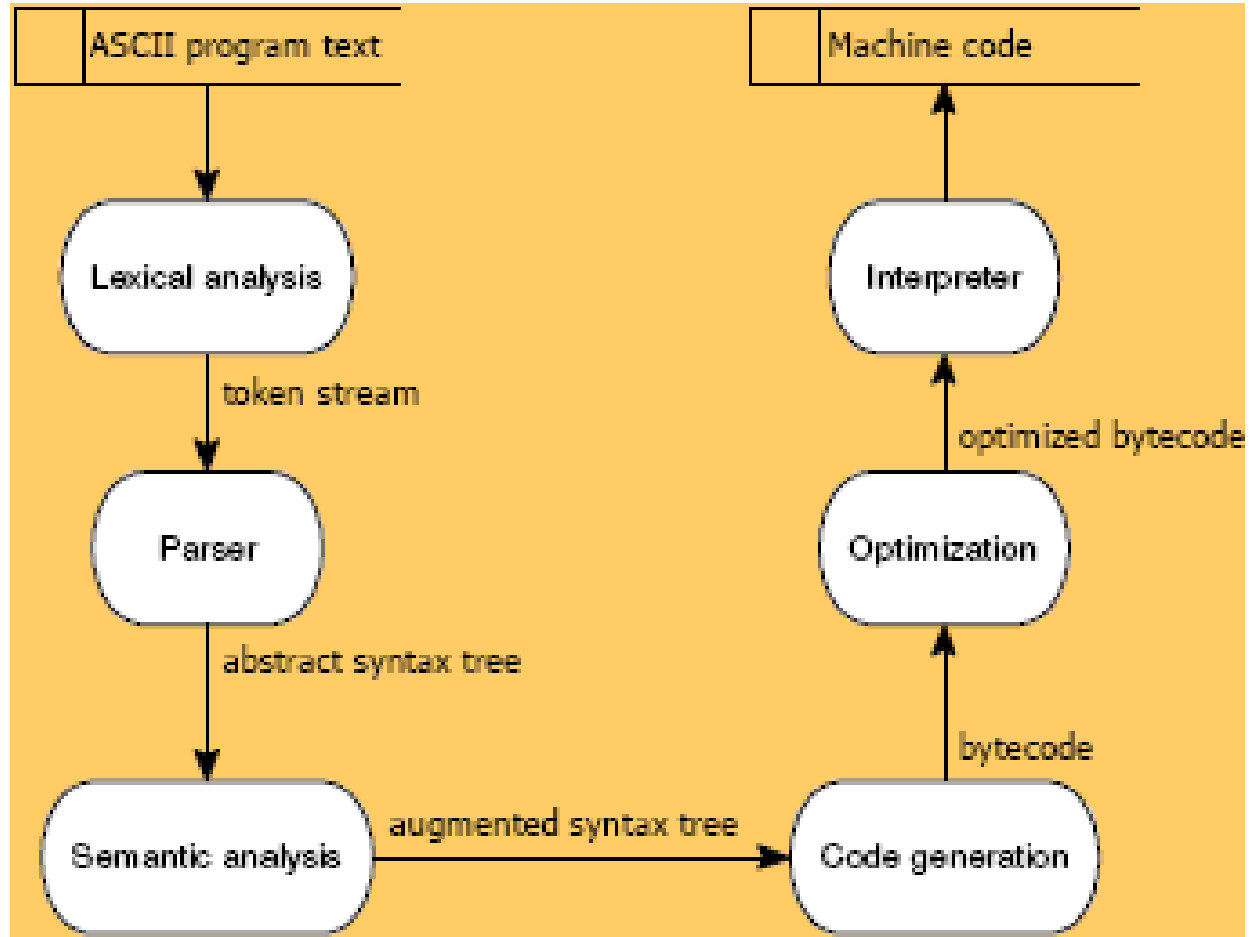
f1.join()

f2.join()

2. Pipe and Filter Architecture (14) - Applications

- **Audio/Video Processing**- Applying effects like noise reduction, equalization, or compression.
- **Stream Processing** - Real-time log analysis and monitoring tools like Splunk.
- **Compilers**- Lexical analysis, parsing, and code generation as separate filters.
- **Data Integration**- Real-time transformation and aggregation of streaming data.

Compilers



2. Pipe and Filter Architecture (15)

- **Benefits**

- Concurrency: All filters may operate at the same time
 - High throughput for excessive data processing
 - Consuming and producing data incrementally
- Reusability: Any two filters can be connected if they agree on that data format that is transmitted
- Flexibility: Supports both parallel and sequential execution
- Simplicity: Offers clear division between any two filters connected by a pipe
- Modifiability:
 - New filters can be added
 - Old filters can be modified as long as I/O interfaces are unchanged

- **Limitations**

- No user interaction (not suitable for dynamic interaction)
- Difficult to configure (make up) pipe and filter dynamically
- Filter ordering crucial
- Multiple I/O
- Difference in arrival data rate might impose a performance bottleneck

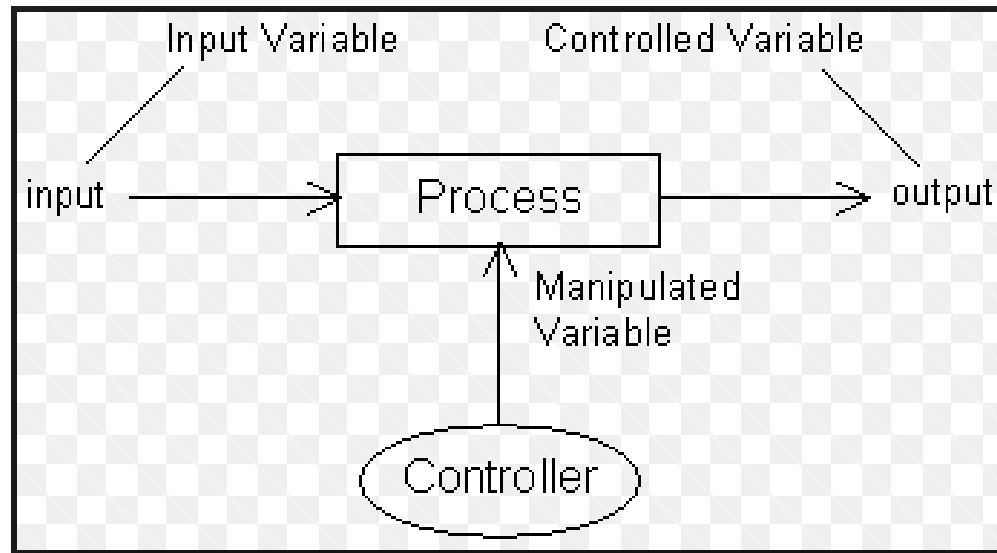
3. Process Control (1)

- The process control architecture is best suited to controlling complex continuous systems that interact with nature or other complex changing environment that makes it difficult to define an algorithm for directly transforming inputs to outputs.
- This pattern is used to maintain the desired state of a system through continuous monitoring and adjustment, often in a feedback loop.
- Ex: Controlling machinery operations like temperature, pressure, and flow in manufacturing plants.

3. Process Control (2)

- It is a type of data flow architecture where data flow is neither batched sequential nor pipelined stream.
- The flow of data comes from a set of variables, which controls the execution of process.
- It decomposes the entire system into subsystems or modules and connects them.

3. Process Control (3)



- The basic elements of process control are a process with inputs and outputs and a controller that has some control over the process and therefore the outputs.

3. Process Control (4)

- Process variables - properties of the system that can be measured (any property of the system that can be measured).
- Process variables include: inputs, outputs, and intermediate states of the system.

3. Process Control (5)

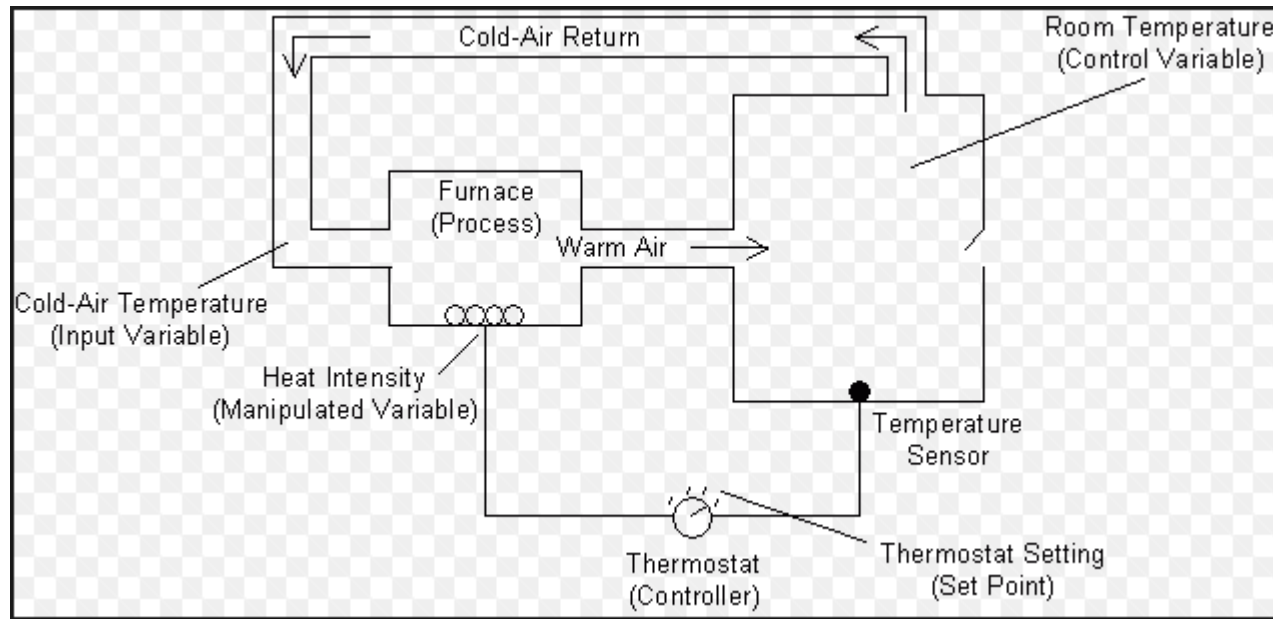
- Input variables: variables that measure inputs to the process.
- Control (output) variables: - variables which the system intends to control.
- Manipulated variables - variables which can be changed by the controller.
- Process – What are you trying to control
- Set point - desired value for control variables
- Sensor – Obtains values of process variables pertinent to control and can be used as a feedback reference to recalculate manipulated variables.
- Controller – is software that manipulate variables
- Control Algorithm – It is used for deciding how to manipulate process variables.

3. Process Control (6) – Example

Room Temperature Control

- **Input Variables** - Outdoor temperature (e.g., a cold day or hot afternoon), Occupancy in the room (more people generate more heat).
- **Control Variables** -Room temperature (the system tries to keep this at a specific level).
- **Manipulated Variables** -Heating or cooling power of the HVAC system.
- **Process** -The thermal dynamics of the room (how the temperature changes with heating, cooling, and external conditions).
- **Set Point** -22°C (desired temperature).
- **Sensor** (temperature sensor, CO₂ sensor)
- **Controller**- The controller software turns the heating or cooling system on or off depending on the room temperature.
- **Control Algorithm** – Software based on PID algorithm

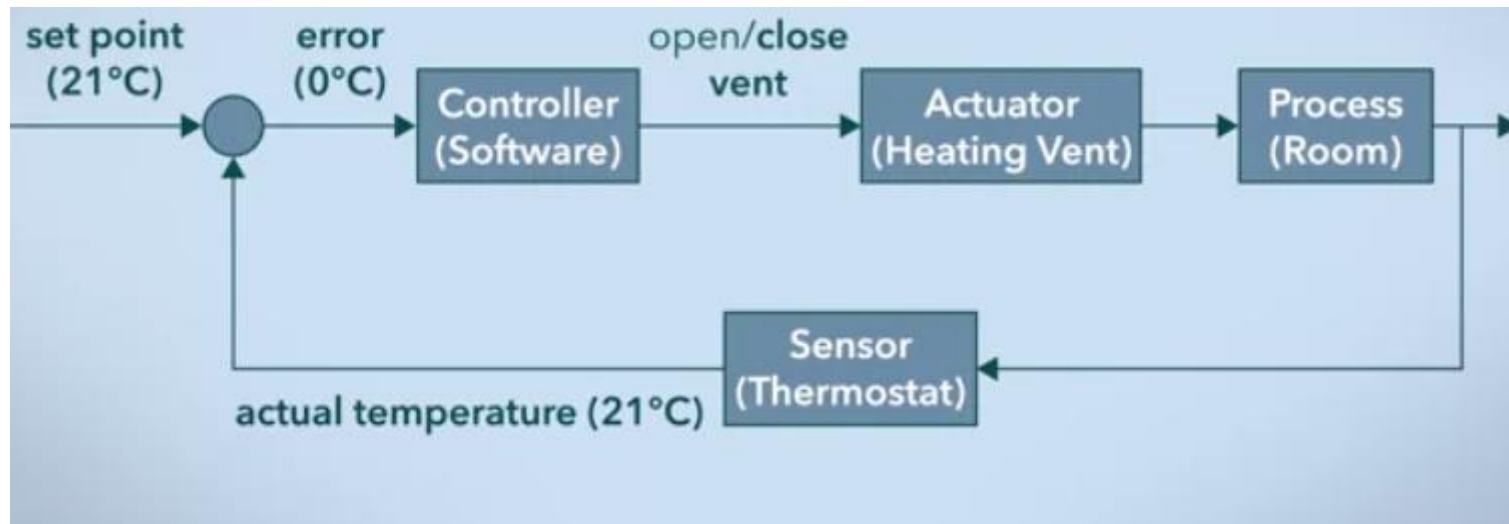
3. Process Control (7) – Example Room Temperature Control



3. Process Control (8) – Closed loop system

- The most common type of process control system is a **closed-loop system**.
- In a closed-loop system the values of process variables (usually input and control variables) are used to adjust the process in order to reach the desired set point.
- A system where feedback is continuously collected, compared with the set point, and used to adjust the process in real-time.
- Ensures that disturbances (external input variables) are accounted for and corrected dynamically.
- Operates autonomously without manual intervention.

3. Process Control (9) – Example Room Temperature Control



3. Process Control (9)- Applications

- Industrial Automation
 - Controlling machinery operations like temperature, pressure, and flow in manufacturing plants.
- Robotics
 - Maintaining balance or following a path using sensors and feedback loops.
- Aerospace
 - Flight control systems to stabilize and navigate aircraft.
- Energy Systems
 - Monitoring and controlling power grids or renewable energy sources for optimal output.